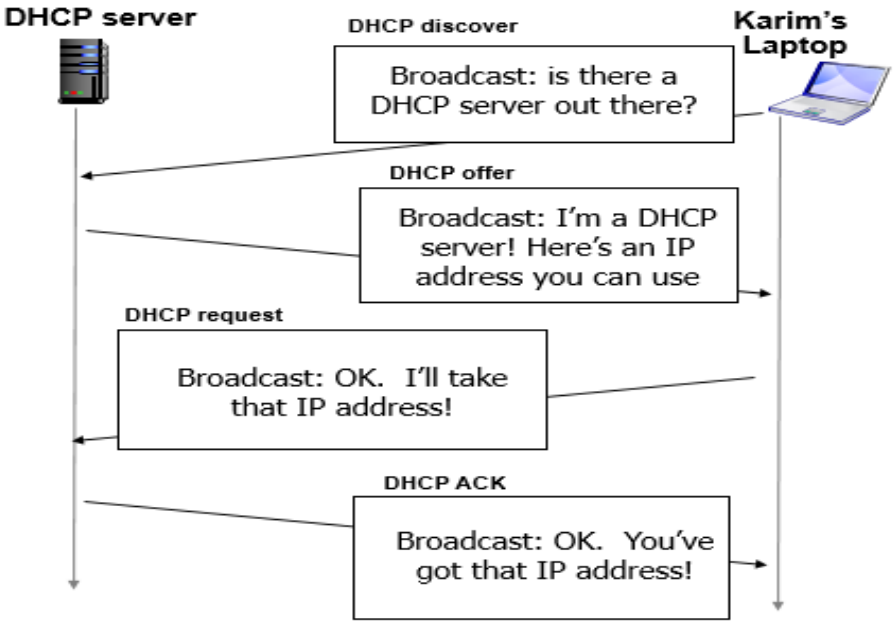
CN Sp24 ReFinal Solution

Q1. a). (10)

Application	Some data loss is ok or not	Is there any minimum throughput required (yes/No)	Time sensitive task? Service with some delay is ok or not.
Uploading an image on a webpage	Ok / No	Yes / No	Ok / No
Video chat on Google Meet	Ok / No	Yes / No	Ok / No
Playing an online interactive game.	Ok / No	Yes / No	Ok / No
Watching a video	Ok / No	Yes / No	Ok / No

b i. (2)



ii. (1)

when karim initiates a query to flex.nu.edu.pk, the host makes DNS query, query is sent to its local DNS server which has local cache of recent name-to-address translation pairs. If not query will be forwarded to TLD Server and Root level servers.

OR

i. *Do a DNS lookup to get IP address of flex domain.*
ii. *Show all HTTP exchanges to get the default home page.*

iii. (2)

This is a similar to the file upload to multiple peer we studied in the class. Client-server model need more computer, storage and network bandwidth as the number of users accessing the shared file folders grows to a large number (video lecture where size is ~1000s of MB).

A P2P based application on each peer (like Bit torrent running on LAN) will allow a new users to use multiple copies of lectures those stored on different peers (as well as server copy) to take part of the download. Each peer PC will provide the bandwidth hence removing the bottleneck.

Q2.. (20)

Time Unit (RTT)	cwnd	State of Congestion window (1.5 mark each)	ssthresh value (1 mark each)
1	8	slow start	8
2	9	congestion avoidance	8
3	7	Fast Recovery	4
4	4	congestion avoidance	4
5	5	congestion avoidance	4
6	5	Fast Recovery	2
7	2	congestion avoidance	2
8	4	Fast Recovery	1
9	8	Fast Recovery	1

Q3. a). (3)

EstimatedRTT = (1-α) * EstimatedRTT + α * SampleRTT
= 0.875x30 ms + 0.125x28 ms = 29.75 ms

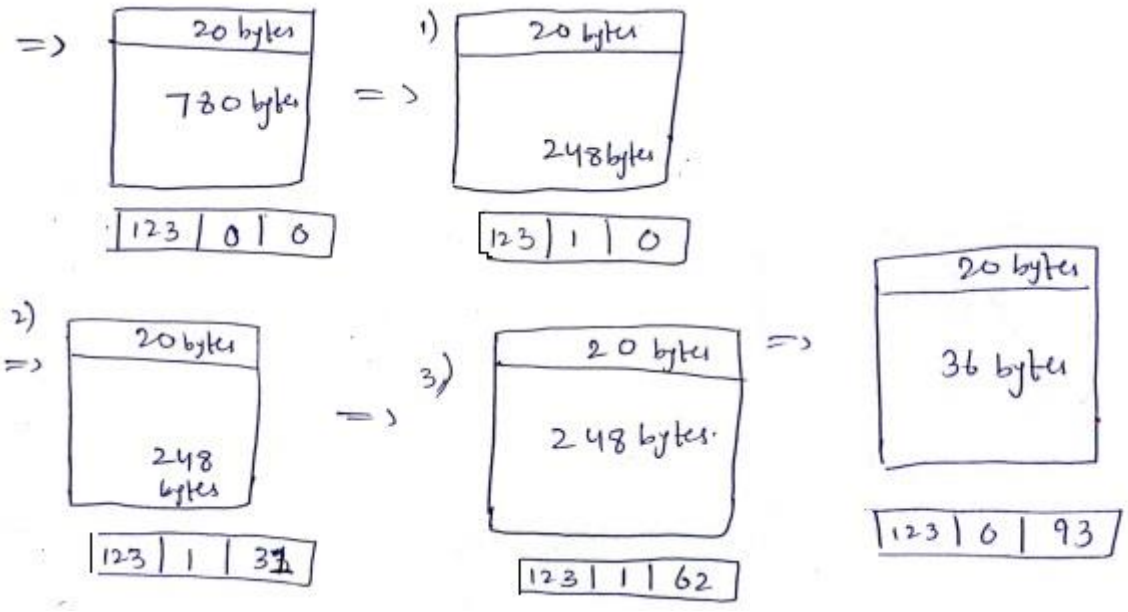
DevRTT = (1-β) * DevRTT + β*|SampleRTT-EstimatedRTT|
= 0.75 x 3 ms + 0.25 x |28 ms – 29.75 ms| = 2.6875 ms

RTO = EstimatedRTT + 4*DevRTT
= 29.75 ms + 4x2.6875 ms = 40.5 ms

b). (2)

No, UDP does not perform retransmissions and, thus, does not require an RTT-based timeout timer.

c). (5)



d). i. (1) Places where infrastructure communication is not possible, like oceans, airplane (space) etc.
ii. (2) Giving more channels to downlink than uplink.
iii. (2). If the signal strength of another signal is under tolerable range, it is noise, else interference.
iv. (2) They will need IPv4 public address for all their hosts.
v. (2) 16 bit, or 64K.
vi: (2) 4 and 6 only
vii. (2) 0 to 4
viii. (2) to specify maximum number of hops.
ix. (2) to specify transport layer protocol, e.g., TCP, UDP etc.
x. (3) Missing fields: Checksum, Fragmentation, Options

e). (5 * 1 = 5)

a) Gateway = 192.168.10.1
b) Network Address R2 Fa0/0 = 192.168.20.0
c) Subnet mask Host 2 = 255.255.255.0
d) No. of Subnets = 3
e) Subnet mask Class B = 255.255.0.0

Q4. a) i. (2)

message complexity
LS: *n* routers, O(n^2) messages sent
DV: exchange between neighbors; convergence time varies

ii. (3)

robustness: what happens if router malfunctions, or is compromised?
LS:
router can advertise incorrect *link* cost
each router computes only its *own* table
DV:
DV router can advertise incorrect *path* cost ("I have a *really* low-cost path to everywhere"): *black-holing*
each router's DV is used by others: error propagate thru network

b). i. (2)

iterative, asynchronous: each local iteration caused by:
local link cost change
DV update message from neighbor

ii. (3)

distributed, self-stopping: each node notifies neighbors *only* when its DV changes

- neighbors then notify their neighbors – *only if necessary*
- no notification received, no actions taken!

c). (3) RIP, EIGRP, OSPF

d). (2) No any such (That is all routers are running iBGP).

Q5. a). i. (2)

token passing:

- control *token* passed from one node to next sequentially.
- token message

polling:
controller node “invites” other nodes (clients)to transmit in turn
typically used with “dumb” devices

ii. (3)
token passing concerns:

- token overhead
- latency
- single point of failure (token)

polling concerns:
polling overhead
latency
single point of failure (controller)

b). (5)

1 0 1 1 | 1
0 1 1 0 | 0
1 1 0 0 | 0
0 0 1 1 | 0
----
0 0 1 0 | 0

c). (5)